MPCA Project

TITLE OF THE PROJECT:Auto-rickshaw fare calculator

Team Members:

Sl. No	Name	SRN
1	Nandhana S	PES1UG24AM440
2	Nishitha Padanthaya	PES1UG24AM442
3	Madhumitha Dhondi	PES1UG24AM923

Components used:

Arduino IDE, Arduino Uno (Microcontroller board), Distance sensor (wheel rotation sensor), IR sensor, GPS Module (NEO-6M), Ultrasonic Sensor (HC-SR04), LCD display (16×2) for showing distance and fare, Push button(start/stop/reset), Breadboard and connecting wires, Power supply,

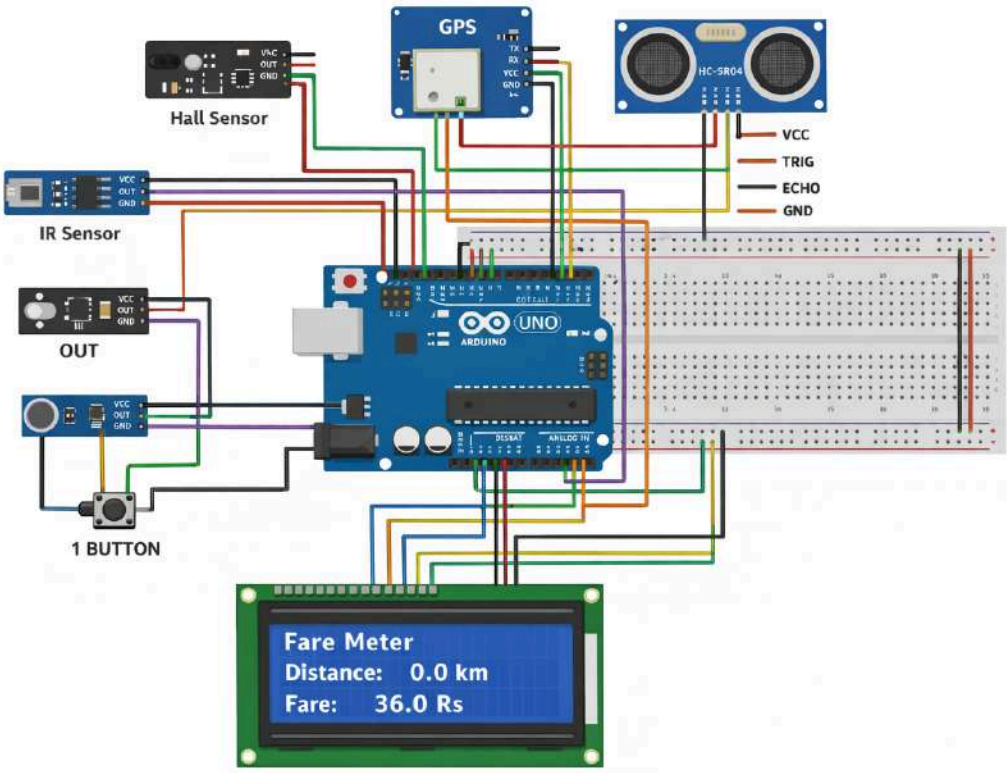
Explanation about the project: Auto Rickshaw Fare Meter using Microcontroller

Project Overview: This project implements an automatic fare calculation system using a microcontroller-based embedded system. It calculates the fare of an auto-rickshaw based on the distance travelled and waiting time, and displays the result on an LCD screen. The system eliminates manual fare calculation errors and improves transparency for passengers.

Objective:

- To automatically calculate fare based on distance.
- To include waiting charges when the vehicle is idle.
- To display distance and fare in real-time.
- To demonstrate sensor interfacing and embedded programming.

Circuit Diagram:



Pseudocode:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>
```

```
// LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);

// GPS
SoftwareSerial gpsSerial(7, 8);
TinyGPSPlus gps;

// Pins
const int hallPin = 2;
const int startBtn = 3;
const int stopBtn = 4;
const int resetBtn = 5;
const int irPin = 6;
const int trigPin = 9;
const int echoPin = 10;
const int ledPin = 13;

// Variables
volatile long pulseCount = 0;
float distance = 0;
float fare = 0;
float waitingFare = 0;
bool rideActive = false;

// Button states
bool lastStart = HIGH;
bool lastStop = HIGH;
bool lastReset = HIGH;

// Settings
unsigned long lastLCDUpdate = 0;
unsigned long waitStart = 0;

// Simulation variables
unsigned long startDelayTime = 0;
unsigned long lastIncrementTime = 0;
```

```
bool delayDone = false;

float speedKmph = 0;

void setup() {
  Serial.begin(9600);

  pinMode(hallPin, INPUT);
  pinMode(startBtn, INPUT_PULLUP);
  pinMode(stopBtn, INPUT_PULLUP);
  pinMode(resetBtn, INPUT_PULLUP);

  pinMode(irPin, INPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);

  attachInterrupt(digitalPinToInterrupt(hallPin), countPulse, RISING);

  lcd.init();
  lcd.backlight();

  gpsSerial.begin(9600);

  lcd.setCursor(0, 0);
  lcd.print("Fare Meter");
  lcd.setCursor(0, 1);
  lcd.print("Ready...");
  Serial.println("System Ready");
  delay(2000);
  lcd.clear();
}

void loop() {

  // ================= BUTTONS =================
  bool startState = digitalRead(startBtn);
```

```
bool stopState  = digitalRead(stopBtn);
bool resetState = digitalRead(resetBtn);

if (startState == LOW && lastStart == HIGH) {
  rideActive = true;
  startDelayTime = 0;
  delayDone = false;
  Serial.println("Ride Started");
  delay(50);
}

if (stopState == LOW && lastStop == HIGH) {
  rideActive = false;
  Serial.println("Ride Stopped");
  delay(50);
}

if (resetState == LOW && lastReset == HIGH) {
  pulseCount = 0;
  distance = 0;
  fare = 0;
  waitingFare = 0;
  speedKmph = 0;

  startDelayTime = 0;
  lastIncrementTime = 0;
  delayDone = false;

  rideActive = false;
  Serial.println("System Reset");
  lcd.clear();
  delay(50);
}

lastStart = startState;
lastStop = stopState;
lastReset = resetState;
```

```
// ================= ULTRASONIC =================
long duration;
float ultraDistance;

digitalWrite(trigPin, LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

duration = pulseIn(echoPin, HIGH, 20000);
ultraDistance = duration * 0.034 / 2;

digitalWrite(ledPin, (ultraDistance > 0 && ultraDistance < 15));

// ================= IR SENSOR =================
int irState = digitalRead(irPin);

// ================= SIMULATION =================
if (rideActive) {

  if (!delayDone) {
    if (startDelayTime == 0) {
      startDelayTime = millis();
    }

    if (millis() - startDelayTime >= 5000) {
      delayDone = true;
      lastIncrementTime = millis();
      Serial.println("Simulation Started");
    }
  }

  if (delayDone && millis() - lastIncrementTime >= 2000) {

    distance += 0.05;
```

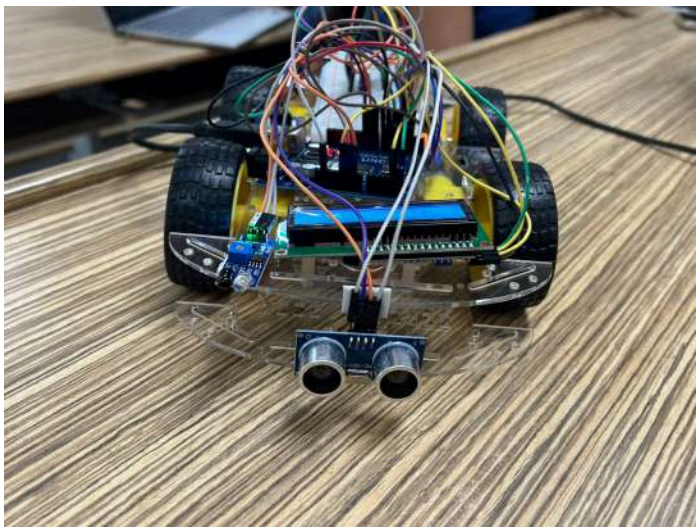
```
    if (speedKmph < 5) {
      speedKmph += 1;
    } else {
      speedKmph = 5;
    }

    lastIncrementTime = millis();
  }

  // ================= FARE =================
  if (distance <= 0.5) {
    fare = 36;
  } else {
    float extra = distance - 0.5;
    int steps = extra / 0.4;
    fare = 36 + steps;
  }

  // ================= WAITING =================
  if (speedKmph < 1) {
    if (waitStart == 0) waitStart = millis();

    if (millis() - waitStart > 5000) {
      waitingFare += 2;
      waitStart = millis();
      Serial.println("Waiting +2");
    }
  } else {
    waitStart = 0;
  }
}

float totalFare = fare + waitingFare;

// ================= SERIAL =================
Serial.print("Ultra: ");
```

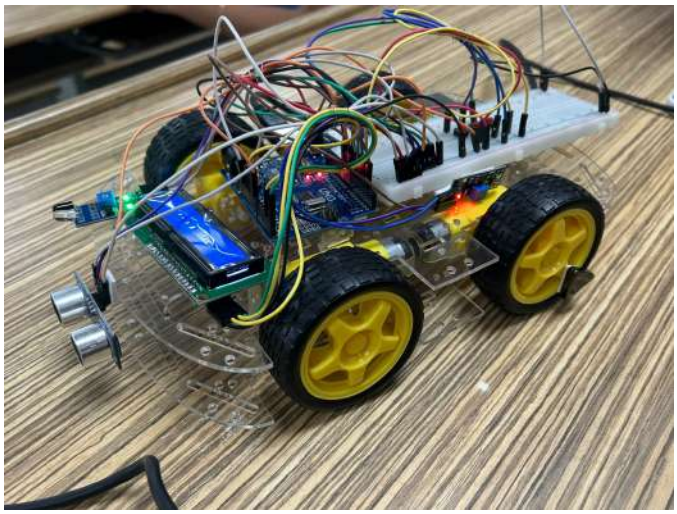
```
  Serial.print(ultraDistance);
  Serial.print(" | IR: ");
  Serial.print(irState == LOW ? "YES" : "NO");
  Serial.print(" | Dist: ");
  Serial.print(distance);
  Serial.print(" m | Speed: ");
  Serial.print(speedKmph);
  Serial.print(" | Fare: ");
  Serial.println(totalFare);

  // ================= LCD =================
  if (millis() - lastLCDUpdate > 300) {
    lcd.setCursor(0, 0);
    lcd.print("Dist:");
    lcd.print(distance, 2);
    lcd.print("m ");

    lcd.setCursor(10, 0);
    lcd.print(rideActive ? "ON " : "OFF");

    lcd.setCursor(0, 1);
    lcd.print("Fare:");
    lcd.print(totalFare, 0);

    lcd.setCursor(8, 1);
    lcd.print("Speed:");
    lcd.print(speedKmph, 0);
    lcd.print("  ");

    lastLCDUpdate = millis();
  }

  delay(300);
}

// ================= INTERRUPT =================
void countPulse() {
```

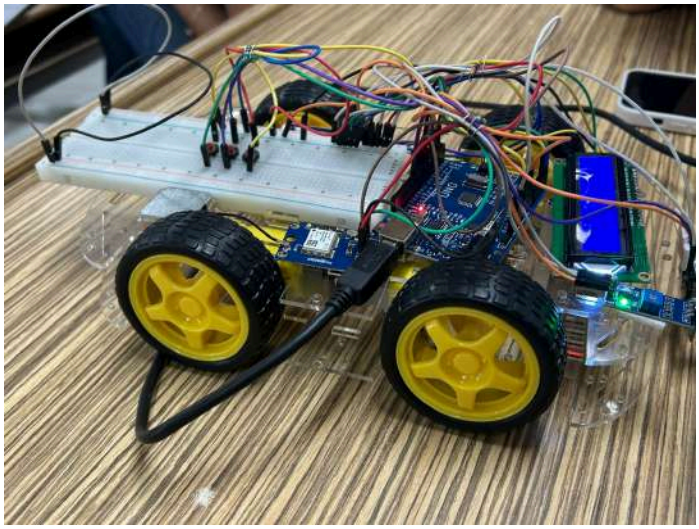
```
  if (rideActive) {
    pulseCount++;
  }
}
```

Output Pictures with explanations:

Model pictures:







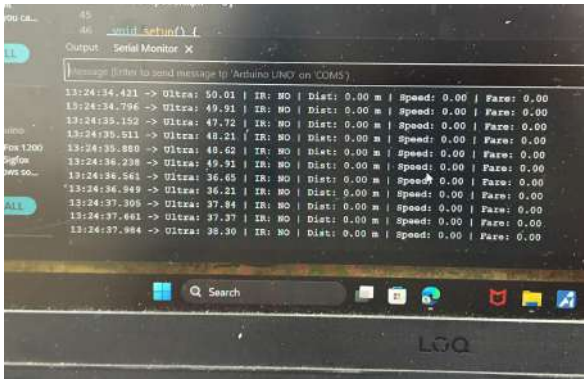
Working:

1) **POWER ON**

- Connect Arduino to power (USB or battery)
- LCD turns ON and shows:

System initializes all components:

- LCD
- Buttons
- Sensors (Hall, IR, Ultrasonic)
- GPS module



2) START BUTTON PRESSED

- When you press the START **button**: Ride begins (`rideActive = true`). System prepares to track distance

DISTANCE MEASUREMENT using Hall Sensor

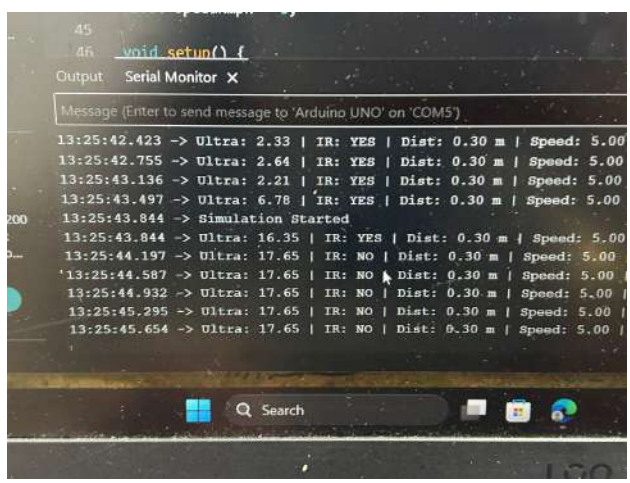
- Magnet attached to wheel rotates
- Hall sensor detects each rotation
- Each pulse = small distance
- More rotations → More distance → More fare

FARE CALCULATION(for demo car)

- First **0.5 meters → ₹36 (base fare)**
- After that: Every 0.4 meters → +₹1
- Formula: Fare = Base + Extra distance charges

SPEED CALCULATION

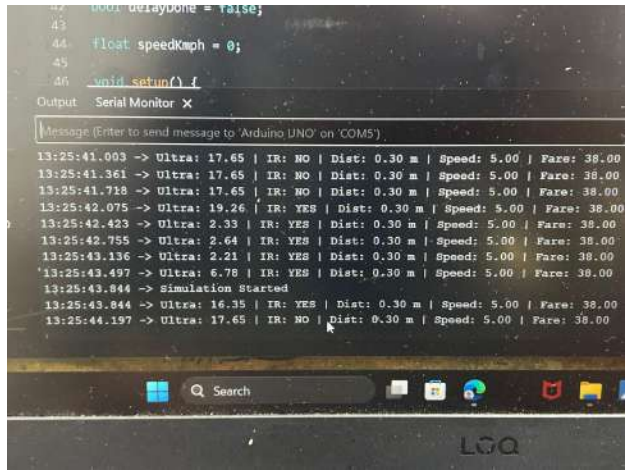
- calculated using the GPS module by reading position data from satellites and determining how fast the location is changing over time.
- TinyGPS++ library processes information to directly provide speed in kilometers per hour using `gps.speed.kmph()`.
- Internally, the GPS calculates speed using the change in position over time or Doppler shift from satellite signals
- to detect waiting time—if the speed drops below a certain threshold (like 1 km/h), the system assumes the vehicle is stationary and adds waiting charges.



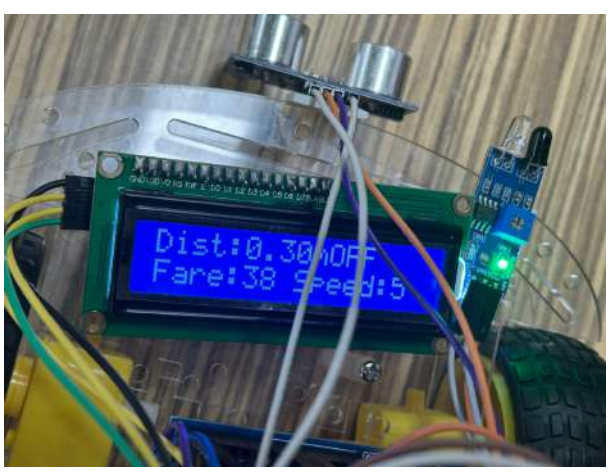
IR and Ultrasonic sensors detecting obstacle and the distance from the obstacle:

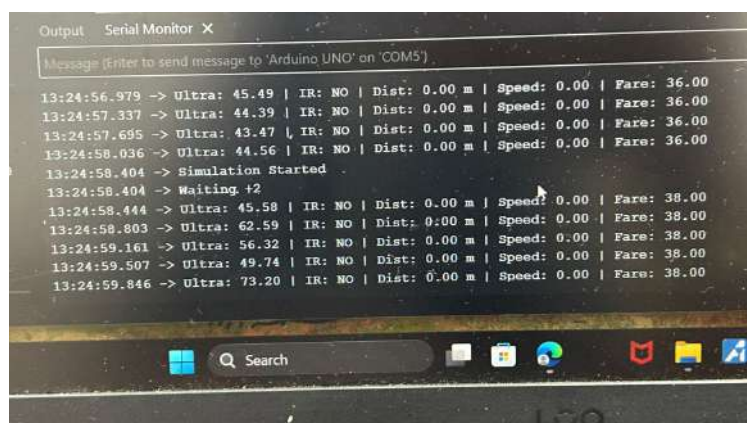
ULTRASONIC SENSOR (Obstacle Detection): Measures distance from object If object < 15 cm: LED turns ON which indicates that an object is detected.

IR SENSOR: Detects nearby objects. If triggered: IR = YES(shown in Serial Monitor)



3) After pushing stop button:





The **STOP button** is used to end the ride without clearing any data, meaning it stops the distance and speed from increasing and freezes the fare at its current value while showing the ride as OFF on the display. In contrast, the **RESET button** completely clears the system by setting distance, fare, speed, and waiting charges back to zero and turning the ride OFF, effectively preparing the system for a new ride.